

Reviewer Pack — Minimal p-Adic Hensel Lifting Steps and Communication Comple...

Entry 6f79ef97da9a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 20:20:56 UTC

Minimal p-Adic Hensel Lifting Steps and Communication Complexity Rank

Entry ID: 6f79ef97da9a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 20:20:56 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): Communication Complexity

Statement:

The number of steps required by the minimal p-adic Hensel lifting algorithm to solve a system of linear equations over the finite field $\mathbb{Z}_p$ is polynomially related to the rank of the communication complexity matrix for that system, such that $|\text{p-adic_Hensel_steps}| = O(f(n))$ where $f(n)$ is a polynomial in n and n is the size of the system.

Rationale (proposer's reasoning):

The p-adic Hensel lifting algorithm provides a constructive approach to solving systems of linear equations, which can be related to the complexity of communication protocols. The polynomial relationship suggests that the efficiency of the algorithm is closely tied to the underlying structure of communication complexity.

Taxonomy category: p-adic_analysis_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): adc618d9bdc729df

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for at least 80% of the systems, the absolute difference between p-adic Hensel lifting steps and the rank of the communication complexity matrix is less than or equal to a polynomially bounded threshold (e.g., ≤ 3), and the correlation coefficient from linear regression analysis is greater than 0.9. The conjecture is falsified if any seed produces a system with an absolute difference exceeding the threshold or a correlation coefficient below 0.7.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'p-adic Hensel lifting' AND 'communication complexity rank' AND 'linear equations' - 'polynomial relation' AND 'p-adic analysis' AND 'communication complexity' - 'finite field $\mathbb{Z}_p$ ' AND 'Hensel algorithm' AND 'algorithmic complexity'

Top relevant hits considered: - [<http://arxiv.org/abs/math-ph/0512018v2>] On Phase Transitions for P -Adic Potts Model with Competing Interactions on a Cayley Tree - [<http://arxiv.org/abs/0910.4839v2>] A p -adic RanSaC algorithm for stereo vision using Hensel lifting - [<http://arxiv.org/abs/2001.06455v3>] A new class of p -adic Lipschitz functions and multidimensional Hensel's Lemma - [<http://arxiv.org/abs/2408.0081>] p -adic Equiangular Lines and p -adic van Lint-Seidel Relative Bound - [<http://arxiv.org/abs/1502.00768v1>] The p -adic analytic subgroup theorem revisited - [<http://arxiv.org/abs/1501.04893v6>] Pro-unipotent harmonic actions and a computation of p -adic cyclotomic multiple zeta values - [<http://arxiv.org/abs/1409.5893v2>] Fast evaluation of far-field signals for time-domain wave propagation - [<http://arxiv.org/abs/2604.22734v1>] Radiation outer boundary conditions and near-to-far field signal transformations for the Bardeen-Press equation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_matrix(n, p):
    A = [[random.randint(0, p-1) for _ in range(n)] for _ in range(n)]
    b = [random.randint(0, p-1) for _ in range(n)]
    return A, b

def gaussian_elimination(A, b, p):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i + random.choice([j for j in range(i, n) if A[j][i] != 0])
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]

        # Eliminate
        for j in range(n):
```

```

        if i != j:
            factor = (A[j][i] * pow(A[i][i], -1, p)) % p
            A[j] = [(A[j][k] - factor * A[i][k]) % p for k in range(n)]
            b[j] = (b[j] - factor * b[i]) % p

    # Back-substitute
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) * pow(A[i][i], -1, p) % p
    return x

def p_adic_hensel_steps(A, b, p):
    n = len(A)
    x = [0] * n
    steps = 0
    while True:
        steps += 1
        x_new = gaussian_elimination(A, b, p)
        if all(x_new[i] == x[i] for i in range(n)):
            break
        x = x_new
    return steps

def communication_complexity_rank(A):
    n = len(A)
    rank = 0
    for i in range(n):
        if any(A[j][i] != 0 for j in range(i, n)):
            rank += 1
            for j in range(i+1, n):
                if A[j][i] != 0:
                    factor = A[j][i] / A[i][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    total_steps = 0
    total_rank = 0
    instances_tested = 0

    for n in n_values:
        A, b = generate_random_matrix(n, p=7)
        steps = p_adic_hensel_steps(A, b, p=7)
        rank = communication_complexity_rank(A)

        total_steps += steps
        total_rank += rank
        instances_tested += 1

    mean_steps = total_steps / len(n_values)
    mean_rank = total_rank / len(n_values)

```

```

diff = abs(mean_steps - mean_rank)
if diff > 3:
    return {
        "metric_name": "p-adic Hensel Steps vs Communication Complexity Rank",
        "metric_value": diff,
        "instances_tested": instances_tested,
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": f"Mean steps: {mean_steps}, Mean rank: {mean_rank}"
    }

return {
    "metric_name": "p-adic Hensel Steps vs Communication Complexity Rank",
    "metric_value": diff,
    "instances_tested": instances_tested,
    "n_max": max(n_values),
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_diff = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_diff} std=0 support_fraction={support_fraction}")
    elif any(r["counterexample"] != "" for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["counterexample"] != "")
        print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)][\"counterexample\"]}\"")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f74c76b.py", line 114, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f74c76b.py", line 79, in run_trial
    steps = p_adic_hensel_steps(A, b, p=7)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f74c76b.py", line 50, in p_adic_hensel_steps

```

```
x_new = gaussian_elimination(A, b, p)
~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f74c76b.py", line 28, in gaussian_elim
A[i], A[max_row] = A[max_row], A[i]
~~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete the required computations and verify the conjecture. | next: Investigate the cause of the crash in the test code and attempt to run the test again. If the issue can be resolved, retest the conjecture according to the pre-registered support conditions.

11. Audit log (LLM calls)

Total LLM calls: 13

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14606
2	propose	ollama_remote	glm4:latest	0	0	11706
3	propose	ollama_remote	glm4:latest	0	0	12269
4	propose	ollama_remote	glm4:latest	0	0	12698
5	propose	ollama_remote	glm4:latest	0	0	18758
6	preregistration	ollama_remote	glm4:latest	0	0	12371
7	novelty	ollama_remote	glm4:latest	0	0	8474
8	novelty	ollama_remote	glm4:latest	0	0	9918
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20878
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11657
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11578
12	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14311
13	judge	ollama_remote	glm4:latest	0	0	12072

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 171297 ms total latency. Provider mix: {'ollama_remote': 13}

(full prompt+response transcripts available in [research/audit/6f79ef97da9a.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/6f79ef97da9a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6f79ef97da9a.tar.gz` (if generated)